

RESEARCH ARTICLE

Performance-Oriented and Sustainability-Oriented Design of an Effective Android Malware Detector

SANA QADIR¹, (Member, IEEE), AMNA NAEEM²,
MEHDI HUSSAIN¹, (Senior Member, IEEE), HUMA GHAFOR¹,
AND AISHA HASSAN ABDALLA HASHIM³, (Senior Member, IEEE)

¹School of Electrical Engineering and Computer Science, National University of Sciences and Technology, Islamabad 44000, Pakistan

²School of Interdisciplinary Engineering and Science, National University of Sciences and Technology, Islamabad 44000, Pakistan

³International Islamic University Malaysia, Kuala Lumpur 50728, Malaysia

Corresponding author: Sana Qadir (sana.qadir@seecs.edu.pk)

ABSTRACT Effective Android malware detection is a complex problem because of the rapidly-evolving, complicated, and diverse nature of malware. The design of malware detectors should prioritise high detection rate, efficient use of computational resources, and sustainability. Keeping these design priorities in mind, we develop and empirically evaluate four different classifiers. Firstly, to ensure high detection rate, we use a dataset compiled using hybrid analysis of a diverse set of apps. Unlike most publicly-available Android datasets, the dynamic analysis of each app was carried out on a real device and not on a virtual setup. This means that this dataset contains a comprehensive profile of sophisticated malware capable of changing its behaviour on a virtual setup. Secondly, to enhance efficiency, we explore the use of a GPU-based setup and different feature selection techniques. Lastly, we emphasize sustainability by training the models using apps that date back to the beginning of the Android ecosystem i.e. from 2008 until 2020. Our results show that Random Forest (RF) is the most effective classifier with the highest accuracy of 97.86%. This accuracy is 2.78% higher than the best accuracy reported in existing literature. The data also shows that RF is the most sustainable classifier with minimal decrease in F1 score for over-time performance. With regard to efficiency, we find that Logistic Regression (LR) is the best option and that the training time of most models improves significantly when a GPU-based setup instead of a CPU-based setup.

INDEX TERMS Malware detection, machine learning, graphics processing unit, performance, sustainability.

I. INTRODUCTION

Nowadays, the most pervasive mobile operating system (OS) is Android. Its prevalence and open-source nature has also made it the most-commonly targeted platform for cyberattacks [1]. 46% of the organizations surveyed in [2] had a mobile-related security breach in 2022. Also, between 2021 and 2022, the number of critical vulnerabilities discovered in Android OS grew from 571 to 897 [3]. Additionally, the total number of unique mobile malware samples rose by 51% with an average of 77,000 unique malware samples detected every month in 2022 [3]. Also, 94% of malware was

polymorphic, meaning that new variants of malware were frequently being released [4]. These statistics highlight the need for effective and sustainable Android malware detectors for safeguarding data, networks, and systems. These systems should be able to efficiently classify an app to either benign or malicious and facilitate real-time response to future malware attacks. It should also be able to quickly and effectively adapt to the rapidly evolving and dynamic nature of the Android threat landscape. Therefore, Machine Learning (ML) and Deep Learning (DL) algorithms are ideal candidates for the design of malware detection systems. Also, they are capable of learning how each type of malware is tailored to impact a targeted system in distinct ways, ranging from enabling remote code execution to stealing confidential data [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Ye Liu¹.

A. STATIC AND DYNAMIC MALWARE ANALYSIS APPROACHES

The performance of an intelligent malware detector directly depends on how comprehensively sample apps have been analysed to understand their structure and behaviour. Fundamentally, this can be through static analysis or dynamic analysis. Static analysis requires obtaining the source code or byte code of an app and extracting characteristic features from it (such as Permissions, API Calls, and Intent Filters). It is a fast approach, in which tools like APKTool,¹ Androguard,² are used to examine files like AndroidManifest.xml and classes.dex [32].

However, static analysis is not an effective method for analysing malware that uses sophisticated mechanisms to evade analysis, such as code obfuscation or encryption. To scrutinise such malware, dynamic analysis (using tools like DroidBox³ and strace⁴) is more effective and it requires running sample apps in a controlled environment such as a virtual machine or sandbox (e.g. CWSandbox⁵) [6]. For example, in [7], the samples were run inside a nested cloud environment and data related to memory usage, network, virtual file system, processes, and kernel-level system calls was extracted [32]. Naturally, dynamic analysis is more time-consuming than static analysis but utilising it is vital for building a comprehensive and robust profile of malicious APKs.

It should also be noted that some recent, sophisticated APKs can detect the presence of a virtual setup. In such cases, the APK does not execute its malicious behaviour. Therefore, researchers consider it more effective to use a real device (instead of a virtual or emulated one) for dynamic analysis.

B. USE OF ML/DL FOR MALWARE DETECTION

The scale, complexity, and evolving nature of the Android ecosystem means that the task of malware detection requires automation and sophisticated algorithms e.g. ML/DL algorithms.

A comprehensively trained ML/DL model should be able to provide real-time detection and minimize false positives. It should also be efficient to re-train or update in order to remain effective at detecting evolving variants of malware.

Although, existing studies on ML-based malware detectors have achieved exceptionally good results (e.g. [8], [9], [10], [11], [12], [13], and [14]), they rely heavily on data collection (via malware analysis) and feature engineering. These steps are time-consuming and give adversaries a chance to acquire knowledge about the features incorporated in a model and find ways to potentially bypass the malware detector [15].

In contrast to ML, DL is able to handle larger amounts of unstructured data without human involvement and it can

achieve higher accuracy and lower false positive rates [16]. Several studies propose DL-based malware detectors that use advancements in automated feature extraction for robust malware detection (e.g. [17], [18] [19], [20]). They also demonstrate exceptional performance in handling intricate patterns and complexities. However, DL models are more complex, require large amounts of data, have significantly longer training time, and offer poor interpretability [4], [21]. The limited resources of mobile systems (that run Android OS) are not suited to on-device training of DL models.

The difficulty of designing an effective and efficient malware detectors is compounded by the fact that cybersecurity is very different from other fields, where AI is generally applied (e.g. computer vision or pattern recognition). Adversaries actively employ evasive techniques and behave in an inconsistent and unpredictable manner. To remain effective (i.e. provide high accuracy), models must be regularly updated or re-trained against new and evolving malware [4]. This phenomenon of decreasing detection rates over time is known as concept drift or model *sustainability*. On average, the performance of a malware detector should fall with respect to time as new malware variants are increasingly different from older variants. Sustainability is a very challenging attribute to ensure for several reasons. There can be a lack of new samples, and it can be time-consuming to comprehensively analyse each sample (and extract relevant features). Lastly, it is also computationally expensive to frequently update / re-train a model.

C. RESEARCH GAP AND OBJECTIVES

Recent studies on malware detectors show very promising results, especially when static and dynamic malware analysis are combined via a *hybrid analysis* approach (such as in [22], [23], [24], [25], [26], [27]). However, these studies do not sufficiently explore the effect of using a GPU (instead of a CPU) and feature selection on efficiency. They also emphasize high accuracy rates (i.e. effectiveness) without adequately investigating sustainability [28]. To the authors' knowledge no such work has been done using a dataset compiled using hybrid analysis.

This paper aims to fill these research gaps by developing a malware detector that has been empirically evaluated to provide the best trade-off in terms of effectiveness, efficiency, and sustainability. Basically, this study has two research objectives:

- **RO-1:** develop an effective and efficient malware detector. Four different ML/DL models are trained and evaluated on a CPU-based setup and then on a GPU-based setup (without feature selection and with feature selection). The model with the best trade-off (especially in terms of accuracy and training time) is recommended.
- **RO-2:** develop a sustainable malware detector. The algorithms used are the same as RO-1 but the test and training sets are created for over-time evaluation of the

¹<https://apktool.org/>

²<https://github.com/androguard/androguard>

³<https://github.com/pjlantz/droidbox>

⁴<https://strace.io/>

⁵<http://cwsandbox.org/>

trained models. The model with the lowest drop in F1 score is recommended.

The rest of the paper is organized as follows. Section II reviews existing ML/DL-based malware detection models with a focus on studies that include a GPU-based setup and on studies that evaluate model sustainability. Section III presents the methodology utilised for each research objective, while section IV presents the results and recommends the best overall models. Section V compares the models with existing malware detectors and section VI concludes the paper with clearly-specified directions for future work.

II. RELATED WORK

This section is divided into three parts. The first part explores how model selection, malware analysis approaches, and feature representation techniques impact the design of malware detectors. The second part reviews work that explores the effect of using a GPU-based setup on malware detectors. The final part looks at studies that gauge model sustainability.

A. IMPORTANT FACTORS FOR DESIGN OF MALWARE DETECTORS

1) SELECTION OF ML, DL, OR ENSEMBLE MODELS

In terms of approach, supervised learning is generally known to provide higher detection rates and lower false positive rates than unsupervised learning [29], [30]. The shallow architecture of supervised ML algorithms, such as Naive Bayes (NB) and Logistic Regression (LR), makes them more efficient classifiers than Decision Trees (DT), Support Vector Machine (SVM), and K-Nearest Neighbour (KNN) ([4], [31], [32]). However, in terms of accuracy, NB and LR perform poorly compared to DT and KNN [4], [31]. Also, compared to ML, DL produces much better results, but at the cost of slower performance and higher resource usage [32], [33]. The most popular DL models for malware detection are Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN) [34]. The former is vulnerable to attacks, while the latter has high computational requirements. CNN is also prone to over-fitting. In terms of performance, the generally agreed upon descending order is RNN, CNN, AE (Auto-Encoder) and LSTM (Long Short-Term Memory) [16].

There is also an increasing interest in improving the performance of DL-based models by using efficient activation functions, loss functions, optimization algorithms, and regularisation methods. For example, Stochastic Gradient Descent (SGD) is recommended as a faster alternative to the traditional Batch Gradient Descent (BGD) because it computes the gradient of the cost function at each epoch instead of the whole dataset (all samples) [35]. In [36], it was used to achieve 94.2% accuracy for the IoTID20⁶ dataset and 73.5% accuracy for the UNSW-NB15⁷ dataset.

The low accuracy, especially in the case of the second dataset, is attributed to the non-linearity of the samples and not to the use of SGD.

Djenna et al. adopted a different approach to improve the performance of their DL-based malware detector [37]. They proposed a novel method that integrated dynamic DL and heuristic methods for detection and classification of advanced and emerging malware. Their results showed that combining heuristic-based and behaviour-based approaches with CNN or Deep Neural Network (DNN) models resulted in superior performance than using only CNN or DNN.

Lastly, ensemble models are also very popular for malware detection. Random Forest (RF), for instance, provides excellent accuracy and reasonable training time. In [39], an Android malware detection framework was created using the bagging method. The framework was trained using permissions extracted via static analysis. Two ensembles were developed and compared with KNN, NB, SVM, and DT. The classifier based on multiple Linear Regression (LR) algorithms showcased notable performance without requiring the use of any complex algorithms. Ma et al. also noted that ensembles which include DL (e.g. DNN and LSTM) can be computationally expensive [38]. Therefore, it is important to carefully select models that are efficient for effective malware detection.

2) SELECTION OF STATIC, DYNAMIC, OR HYBRID APPROACH FOR MALWARE ANALYSIS

There are some static analysis-based studies that have achieved excellent detection rates. For example, in [41], a lightweight and efficient solution for real-time Android malware classification was developed using RF. The best F-score of 95.1% was achieved by the source code-based model, while the permission-based classification model achieved a F-score of 89%. In [42], an efficient malware detector and family classifier named *DroidSieve*, was developed. It was also trained using features extracted via static analysis and focused on obfuscated samples to achieve 99.82% accuracy for malware detection using Extra Tree (ET).

For more effective detection (of sophisticated malware), it is vital to employ dynamic malware analysis techniques that capture a more comprehensive profile of malware apps. In [40], a distributed malware analysis environment was developed by extending the Cuckoo Sandbox.⁸ A RF classifier was implemented (with a unique combination of features) to achieve a weighted average AUC (Area Under Curve) of 0.98. In [12], a data mining approach for behavioural malware detection was proposed. It used a new dynamic analysis method and employed various classification algorithms (including Neural Network, BayesNet, J48, and SVM). The best performance was obtained by the logistic regression detector. This study also highlights the effectiveness of simple models.

⁶<https://sites.google.com/view/iot-network-intrusion-dataset/home>

⁷<https://research.unsw.edu.au/projects/unsw-nb15-dataset>

⁸<https://cuckoosandbox.org/>

More recently, the limitations of using only static or dynamic analysis has led to an increase in adoption of hybrid analysis approach. In [43], a two-level classifier framework was proposed using a dataset compiled from static and dynamic analysis of 220 sample files. Several algorithms, including J48, RF, and Sequential Minimal Optimization (SMO) were evaluated to obtain state-of-the-art detection and classification accuracy [8]. Likewise, in [44] a hybrid analysis-based dataset was used to train traditional ML models for effective malware detection (98.03%) and category classification (87.56%).

3) SELECTION OF FEATURE REPRESENTATION METHOD

Another factor that has a significant impact on performance is feature extraction and representation. Complex representations, such as images and graphs, are more common than simple vector-based representations for DL models because they are capable of modelling temporal or spatial relationships between features (e.g. in [38], [45], [46]). In a very interesting study [47], a comparison of feature extraction time and pre-processing time was presented for several static analysis techniques. The fastest time of 4 minutes 47 seconds was reported for string-based static analysis. The next fastest time was 14 minutes 19 seconds for image-based static analysis. However, because the former used a traditional ML classifier, the prediction time was less than 13 seconds compared to more than 2 minutes required by the DL model used in the latter case. This study clearly makes the case for preferring simple representation methods and ML models for real-time prediction.

B. STUDIES THAT EXPLORE THE EFFECT OF GPU-BASED SETUPS ON MALWARE DETECTION MODELS

Graphics Processing Unit (GPU) architectures offer significant computational advantages (e.g. speed of analysing large datasets), that can improve the overall performance of malware detection systems.

Sahay et al. proposed an efficient malware detection solution that performed parallel classification using general-purpose GPUs [48]. The authors used NB and demonstrated a substantial improvement in detection time. Similarly, in [49], the effectiveness of LR for detection of Android ransomware on different setups was evaluated. The trained model achieved an accuracy of 99.59% and had a detection time that was 60 seconds faster because of the GPU setup. GPU setups have also been used for DL-based malware detectors. In [50], a novel approach using a parallel CNN architecture was presented. The specifications of the setup included an Intel Core i7 8700k with 12 threads, 32.0 GB of RAM, and a GeForce GTX 950 GPU with 2 GB of VRAM. The resulting model was very efficient with detection rates that outperformed traditional ML approaches.

The last few years has seen the update of TensorFlow Core⁹ for GPU environments and the release of cuML¹⁰ for running traditional tabular ML tasks on GPUs. Using the former, Jeon et al. proposed a SAT (Static Analysis with TensorFlow) scheme that developed a DL model using malware signatures [51]. Motylinski et al. utilised GPU-accelerated RF, kNN, SVM, and LR from the cuML library to detect different types of botnet attacks [4]. The authors highlighted the importance of GPU-acceleration to significantly reduce training and estimation time. Likewise, in [52], a GPU-based setup was used to enhance the efficiency of a Deep CNN (DCNN)-based framework for intrusion detection. It achieved a detection accuracy of 99.79%-100% and addressed the limitations of traditional methods in terms of handling complex threats. Moreover, Waqar et al. developed a real-time multi-class Android-based IoT (AIoT) malware detection approach by integrating three different DL models [53]. The resulting hybrid model was trained using CICAndMal2017¹¹ dataset and delivered exceptionally high detection accuracy. Thus, it is obvious that GPU-based setups can significantly improve efficiency of DL models. Table 1 presents a summary of the three most important studies mentioned above. It should be noted that although only [53] focuses on Android, it does not use hybrid approach and only uses computationally intensive DL algorithms.

C. STUDIES THAT EXPLORE THE EFFECT OF TIME ON MALWARE DETECTION MODELS

There are limited studies on the sustainability of malware detection models. To begin with, it is difficult to find accurate temporal information, such as timestamps, for malicious and benign samples. Secondly, it is difficult to determine or define the characteristics of an app that change over time.

In [54], the evolution of static and dynamic characteristics of benign apps over the years 2010-2017 was investigated. Unfortunately, it is not possible to generalise these findings to malicious apps. In another study [55], the dynamic evolutionary behaviour of benign and malicious Android apps was examined using a dataset of 15,451 benign samples and 15,183 malware samples. It derived several important metrics, based on certain patterns, that can help differentiate between benign and malicious apps. However, the effectiveness of using these patterns for detection of rapidly-changing malware were not sufficiently explored. Moreover, this study had the limitation of relying only on dynamic analysis conducted using a virtual setup. In [56], the evolution of malware was explored using differential static analysis. The study succeeded in uncovering several characteristics of malware that changed with respect to time. These changes were often related to new techniques for evading detection.

⁹<https://www.tensorflow.org/guide/gpu>

¹⁰<https://github.com/rapidsai/cuml>

¹¹<https://www.unb.ca/cic/datasets/andmal2017.html>

TABLE 1. State-of-the-art malware detection studies on GPUs.

Source	Task	Dataset	Approach	Model(s)	Feature Selection	Implementation	Best Results
[4]	Botnet Detection	Own subset of Bot-IoT ¹²	Dynamic: Network (PCAP)	RF, kNN, SVM, LR	Permutation Importance	cuML	RF: Accuracy = 0.999 Recall = 0.997 Training Time = 0.45 s
[52]	Network Intrusion Detection	ISCX-IDS 2012 ¹³ DDoS (Kaggle) ¹⁴ CICIDS2017 ¹⁵ CICIDS2018 ¹⁶	Dynamic: Network	Deep CNN (DCNN)	-	TensorFlow	Accuracy = 99.79-100% Training Time = 10-40 s Inference Time = 7.11-29.36 s Model Size: 313 MB
[53]	Android-based IoT Malware Detection	CICAndMal2017 ¹⁷	Dynamic: Network API Calls	LSTM, GRU ^a , BLSTM ^b , Hybrid BLSTM-GRU	-	TensorFlow cuDNN ¹⁸	Hybrid BLSTM-GRU: Accuracy = 99.87% F1 Score = 99.9 Testing Time = 1.38 s

¹Gated Recurrent Unit²Bidirectional Long Short-Term Memory (BLSTM)

One of the earliest and most comprehensive studies on sustainability of malware detectors was done by Xu et al. in 2019 [26]. It proposed a novel classification method based on online learning called *DroidEvolver*. Training was carried out using API calls collected by statically analysing 34,722 malicious and 33,294 benign apps from 2011 to 2016. The average F-score of *DroidEvolver*, for over-time detection ranged from 87.17% to 92.32%. Because this model was trained using statically extracted features, it is limited in its ability to detect sophisticated malware that must be analysed dynamically. Moreover, the apps used in the test set are now more than seven years old.

In [27], Cai proposed an Android malware detection model called *DroidSpan*, using RF. The dataset used was compiled by dynamically analysing 26,382 samples (13,627 benign apps and 12,755 malware) from 2011-2016. The trained model outperformed five state-of-the-art methods and achieved an average F-measure that ranged between 87.17-92.32% for over-time detection. The main limitation of this work, apart from the use of samples that are more than seven years old, was the use of an Android emulator (Nexus One) for dynamic analysis. Analysis carried out on an emulator does not capture the full profile of malware because some malicious apps can detect their presence and refrain from executing their malicious actions.

Ceschin et al. comprehensively evaluated concept drift using two datasets that contained samples from 2009 to 2018 [28]. These datasets were:

- 1) DREBIN¹⁹ consisting of ten textual features (API calls, permissions, URLs, etc.) from 123,453 benign and 5,560 malicious Android applications, and

- 2) AndroZoo²⁰ consisting of eight textual features (resources names, source code classes and methods, manifest permissions, etc.) from 267,342 benign and 80,102 malicious apps.

They developed different drift detectors using Adaptive RF and SGD to obtain an accuracy of 99.00% (for DREBIN dataset) and 89.49% (for AndroZoo dataset). Although, these results are impressive, the features were compiled using static analysis on samples from 2009-2018. The use of only static features means that their malware detector will fail to handle sophisticated malware that can only be comprehensively investigated using dynamic techniques. Table 2 summarises the three above mentioned studies.

III. METHODOLOGY

The methodology consists of two phases. In Phase 1, a sequence of steps is followed to achieve RO-1. Then, in Phase 2, another series of steps are employed for RO-2. The steps common to both phases are described in section III-A, while the steps unique to each phase are detailed in section III-B and section III-C. Figures 1 and 2 illustrate the steps in each phase.

A. COMMON WORKFLOW

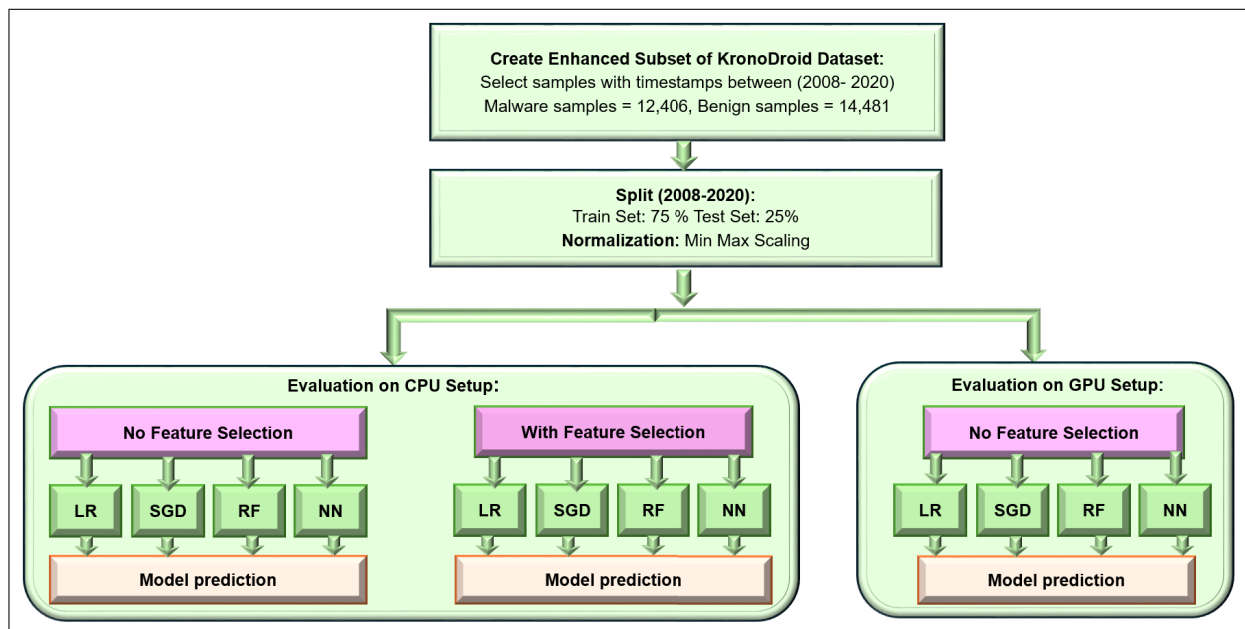
1) SELECTION OF DATASET

To analyse the effectiveness and sustainability of a model, a dataset compiled using hybrid analysis is required. Also, the dynamic analysis should have been carried out on a real device instead of an emulator or virtual setup. Furthermore, it should contain reliable temporal information for each app. This is essential to quantify the degradation in performance

¹⁹<https://www.kaggle.com/datasets/likhithadurusoju/drebindataset>²⁰<https://androzo.uni.lu/>

TABLE 2. State-of-the-art malware detection studies on sustainability.

Source	Task	Dataset	Approach	Model(s)	Feature Selection	Years	Best Results
[26]	Android Malware Detection	B:33,294 M:34,722	Static	Online learning	-	2011-2016	Avg. F-measure = 96.15% (same-time period) Avg. F-measure = 87.17-92.32% (over-time detection)
[27]	Android Malware Detection	B:13,627 M:12,755	Dynamic	RF	-	2011-2017	F-score = 93.88% (same-period detection) Avg. F1 Score = 72% (over-time detection) Feature Time = 351.1 s Learning Time = 0.01 s Storage Cost = 21.2 KB
[28]	Android Malware Detection	DREBIN, AndroZoo	Static	Adaptive RF (A-RF), SGD	-	2009-2018	DREBIN with SGD: Accuracy = 99.00% Avg. F1 Score = 89.19%

**FIGURE 1.** RO-1: Methodology for Phase 1.

by training the model using older samples and testing it on newer samples [57]. In [26], the packaging time included in the dex file is used as temporal information, while in [28] the VirusTotal²¹ submission time was used. Some researchers think that these sources of temporal information are not reliable indicators of time.

A suitable dataset should be derived from sample apps that span over many years so as to adequately capture relevant trends and app evolution. Finally, the dataset should

be representative of the real-world and include different categories of malicious and benign apps.

The only recent Android dataset that meets all these requirements is the KronoDroid²² dataset. The CCCS-CIC-AndMal-2020²³ dataset also meets most of these requirements except that it does not provide timestamps for each sample. The publicly-available CSV of the KronoDroid dataset contains two timestamps i.e. EarliestModDate and

²¹<https://www.virustotal.com/gui/>

²²<https://github.com/aleguma/kronodroid>

²³<https://www.unb.ca/cic/datasets/andmal2020.html>

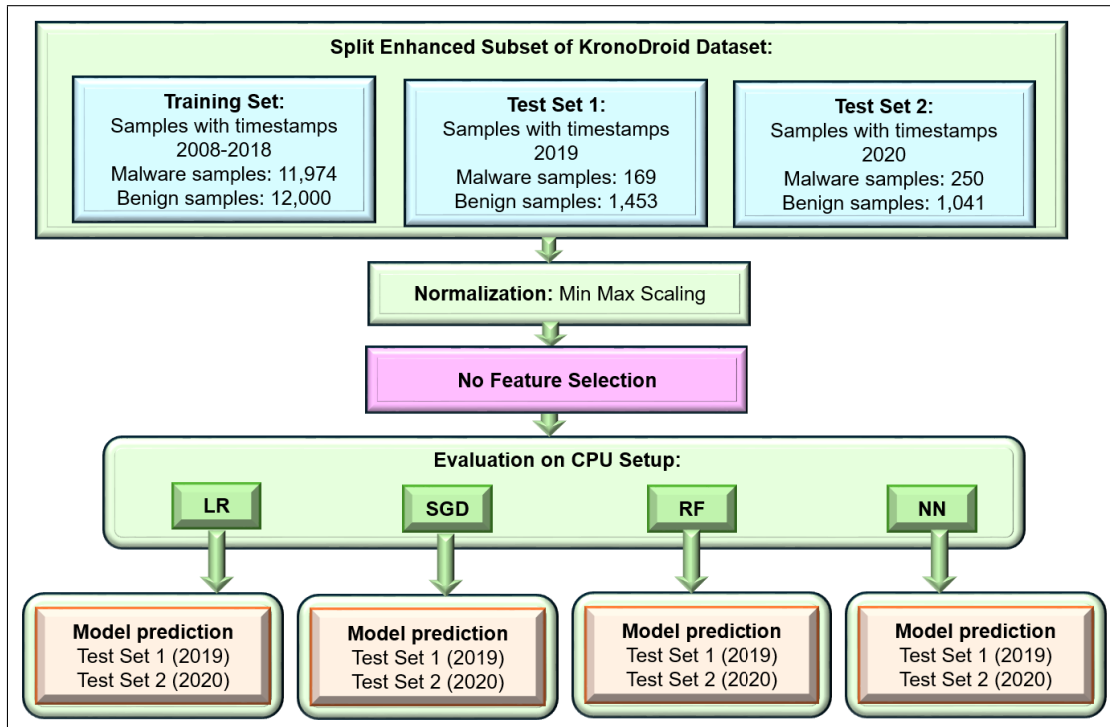


FIGURE 2. RO-2: Methodology for Phase 2.

HighestModDate, with the latter being a more reliable indicator of the time frame of a sample [59].

The KronoDroid dataset was specifically designed for examining concept drift and cross-device detection issues [58]. As such, it was compiled after hybrid analysis of 41,382 malicious apps and 36,755 benign apps (with samples taken from 2008 to 2020). It contains 200 static features and 289 dynamic features and was released in two versions. In the first version, dynamic analysis was performed using an emulator, while in the second version, each app was run on a real device. We only use the second version of this dataset because its analysis setup will ensure it represents sophisticated malware capable of detecting the presence of a virtual setup. In [44], a subset of the second version of KronoDroid was created by adding category labels for the malicious apps (available at [60]). This subset was selected for use in this research.

2) DATA PREPROCESSING

This step typically includes removing null values and data normalization. The former is important to prevent the inclusion of samples with missing values and the latter for improving performance.

To begin with, any sample with a missing or invalid timestamp (e.g. not in the range of 2008-2020) was removed. This reduced the number of samples from 78,137 to 26,887 (B:14,481 and M:12,406). Although there is a slight imbalance in these two categories, it is unavoidable as valid timestamps are essential for the purpose of this study. The

year-wise distribution of these samples is shown in Fig. 3 and the category distribution is shown in Fig. 4.

Originally, this dataset has 485 features. Ten features were dropped (i.e. Package Name, sha256, EarliestModDate, HighestModDate, TimesSubmitted, NrContactedIps, Scanners, Detection_Ratio, MalFamily, Categories) because they were either unreliable or not relevant [58]. One categorical feature representing temporal information was added by extracting the year from the value in the HighestModDate feature. The resulting 476 features are:

- 288 system calls. These are static, numeric features that represent the absolute frequency of each syscall issued by the app at run-time (see Table 3).
- 166 permissions. These are dynamic, binary features that indicate whether an Android permission was requested (i.e. 1) or not requested (i.e. 0) by the app (see Table 3).
- 22 others. These are static, numeric features whose details are shown in Table 4.

A simple vector-based representation is used because it is excellent for efficiency.

3) MODEL IMPLEMENTATION

To maximise variety, the following algorithms were selection:

- two traditional algorithms. The first one is LR (Logistic Regression) and the second one is the probabilistic algorithm, SGD. LR was selected because it is simple and efficient. Likewise, SGD is well-known for its efficiency.

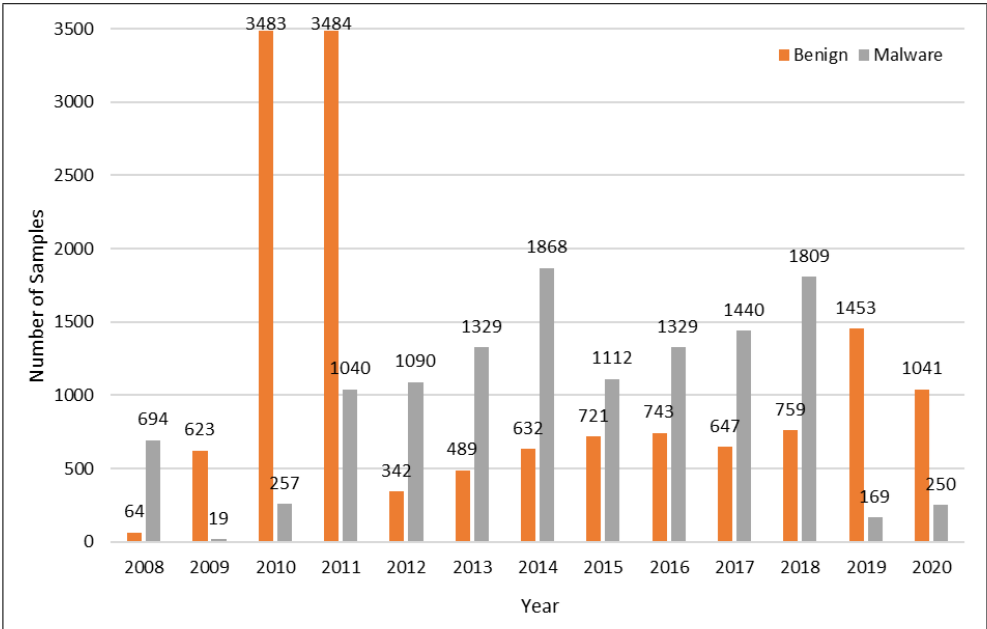


FIGURE 3. RO-1: Distribution of years in enhanced dataset.

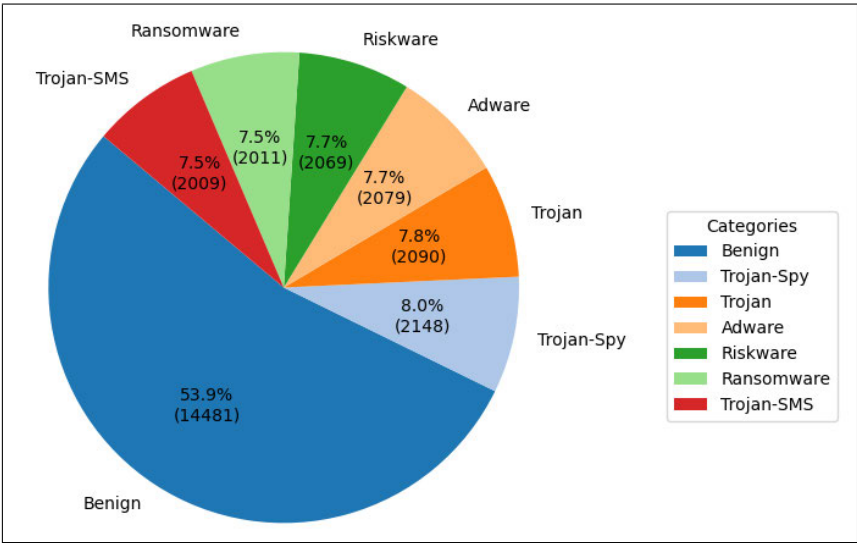


FIGURE 4. RO-1: Distribution of categories in enhanced dataset.

- one ensemble algorithm (RF). This selection was based on RF's excellent accuracy and low false positive rate. It is also exceptional in its ability to capture non-linear relationships and handle complex feature interactions.
- one DL algorithm (NN). This was selected because it is one of the most efficient DL models and it has the capacity to learn intricate relationships between features.

4) MODEL EVALUATION

Each model was evaluated using multiple metrics. Overall, the predictions generated by a model can be grouped into four categories:

- *True Positive* (TP) - the number of instances in which the actual classification is positive and the predicted classification by the model is also positive.
- *False Positive* (FP) - the number of instances in which the actual classification is negative and the predicted classification by the model is positive.
- *True Negative* (TN) - the number of instances in which the actual classification is negative and the predicted classification by the model is also negative.
- *False Negative* (FN) - the number of instances in which the actual classification is positive and the predicted classification by the model is negative.

TABLE 3. Summary of system call (Static) and permission (Dynamic) features.

No.	Type	Category	Name of Feature	Data Type
1.	Static	System Call	execve	Numeric
2.			getuid32	
...			...	
287.			SYS_368	
288.			SYS_369	
290.	Dynamic	Permission	ACCEPT_HANOVER	Categorical
291.			ACCESS_BACKGROUND_LOCATION	
...			...	
454.			WRITE_SYNC_SETTINGS	
455.			WRITE_VOICEMAIL	

Collectively, these categories constitute the *Confusion Matrix*, and are used to calculate the following evaluation metrics:

Accuracy (ACC) - represents the model's overall performance across all classes or categories. It is calculated by dividing the number of correct predictions by the total number of predictions.

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (1)$$

False Positive Rate (FPR) - quantifies the proportion of actual positive cases that were incorrectly identified or classified as positive.

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN} \quad (2)$$

F1 Score - this is defined as the harmonic mean of precision and recall. It is recommended for unbalanced data.

$$\text{F1 Score} = \frac{2 \times \text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}} \quad (3)$$

where

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

and

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

The use of accuracy and FPR is crucial for assessing the effectiveness of malware detectors because incorrect predictions are extremely detrimental for security. The mapping of each metric to the criteria used to assess a model is shown in Table 5.

B. WORKFLOW SPECIFIC TO PHASE 1

The four models are implemented:

- without feature selection on a CPU-based setup.
- with feature selection on a CPU-based setup.
- without feature selection on a GPU-based setup.

1) FEATURE SELECTION

The aim of this step is to choose an optimal subset of features that provide high accuracy and minimise the risk of over-fitting [61]. Popular feature selection algorithms include Information Gain, Chi-square, and Principal Component Analysis [4], [32], [65], [66].

Feature selection can be useful for reducing resource usage, such as training time, by excluding features that do not contribute to the predicted value. However, this must be done with caution because a feature that is removed because it is not considered important today can be important for the detection of future variants. In other words, feature selection can reduce model sustainability, especially in the case of a zero-day attack, a new malware category, or a new malware

TABLE 4. Summary of other (Static) features.

No.	Related to	Name of Feature	Data Type	Description
289.	System Calls	nr_syscalls	Numeric	Total number of syscalls issued by the app at run-time
456.	Permissions	nr_permissions	Numeric	Total number of normal, dangerous and signature permissions requested
457.		normal	Numeric	Total number of normal permissions requested
458.		dangerous	Numeric	Total number of dangerous permissions requested
459.		signature	Numeric	Total number of signature permissions requested
460.		custom_yes	Categorical	Binary feature that indicates if any custom permission was declared by the app (i.e., 1 if any, 0 if none)
461.		nr_custom	Numeric	Total number of custom permissions defined by the application
462.		total_perm	Numeric	Total number of requested permissions. Sum of standard (nr_permissions) and custom (nr_custom) permissions
463.	File	CFileSize	Numeric	Compressed (apk) filesize of the app
464.		UFileSize	Numeric	Sum of the size of all the raw apk inner files)
465.		FilesInsideAPK	Numeric	Number of files inside the apk container (i.e., inner files)
466.	Activities	Activities	Numeric	Total number of activities defined by the application
467.	Intent Filters	NrIntServices	Numeric	Total number of services defined in intent filters
468.		NrIntServicesActions	Numeric	Total number of actions related to services defined in intent filters
469.		NrIntActivities	Numeric	Total number of activities defined in intent filters
470.		NrIntActivitiesActions	Numeric	Total number of actions related to activities defined in intent filters
471.		NrIntReceivers	Numeric	Total number of receivers defined in intent filters
472.		NrIntReceiversActions	Numeric	Total number of actions related to receivers defined in intent filters
473.		TotalIntentFilters	Numeric	Total number of intent filters declared by the application
474.	Services	NrServices	Numeric	Total number of services defined by the application
475.	Time	Year	Categorical	Feature that indicates temporal information about the app (extracted from HighestModDate; in the range of 2008 to 2020)
476.	Label	Malware	Numeric	Binary feature that indicates maliciousness of app (i.e., 1 if yes, 0 if none)

TABLE 5. Metrics.

No.	Metric	Effectiveness	Efficiency	Sustainability	Library	Details
1.	Accuracy (ACC)	✓		✓	sklearn.metrics	accuracy_score (test_labels, predicted_labels)
2.	False Positive Rate (FPR)	✓		✓	sklearn.metrics	FP/(FP+TN)
3.	F1 Score	✓		✓		
4.	Model Size (MS)		✓		joblib	joblib.dump()
5.	Training Time (TT)		✓		time	start_time = time.time() model.fit() training_time = time.time() – start_time
6.	Prediction Time (PT)		✓		time	start_time = time.time() model.predict() Prediction_time = time.time() – start_time

variant. This is reason for feature selection being used in RO-1 only (and not in RO-2).

Out of the three main approaches to feature selection, the filter methods are known to be the most efficient, and therefore, best suited for a model that prioritises real-time performance. The other two approaches, namely wrapper methods and embedded methods, are known to be computationally expensive for large, high-dimensional datasets [62]. The two filter methods that are known to provide good performance for malware detection are the ExtraTree Classifier and Mutual Information algorithms [64].

As a starting point, we selected and implemented multiple feature selection methods for each of the four models. Then, we chose two methods that resulted in the best performing model.

For feature selection, the VarianceThreshold() transformer and SelectFromModel() meta-transformer were chosen for LR, SGD, and RF [63]. The former simply removes all features whose variance is below a given threshold. In the latter method, features are considered unimportant and removed if their importance value is below the provided threshold parameter. Also, it is possible to use the max_features parameter to set a limit on the number of features to select.

For NN, SelectKBest and Principal Component Analysis (PCA) were selected [63]. SelectKBest works by selecting the best features based on uni-variate statistical tests. The k parameter can be used to specify the number of high scoring features to keep and the scoring function can also be specified. PCA offers dimensionality reduction and seeks to replace n features with their k uncorrelated linear combinations, where $k < n$.

2) MODEL TRAINING, PREDICTION, AND EVALUATION

The traditional train_test_split() function is used to split the dataset into train (75%) and test (25%) sets. The libraries,

modules, and hyperparameters used are shown in Table 6 and the code is available at [67]. Table 6 also lists the number of static (S) features and dynamic (D) features selected by the feature selection method. For reference, the complete list of 88 selected features has also be uploaded at [67].

For model evaluation, the following metrics are used to assess resource usage (computational time and memory) and quantify the benefit of using a GPU-based setup instead of a CPU-based setup:

- Accuracy (ACC)
- False Positive Rate (FPR)
- F1 Score
- Model Size (MS) in MB
- Training Time (TT) in seconds - the time taken to train the model using the training set.
- Prediction Time (PT) in seconds - the time taken to generate predictions using the test set.

C. WORKFLOW SPECIFIC TO PHASE 2

1) MODEL TRAINING, PREDICTION, AND EVALUATION

In phase 2, the train and test sets are created based on the year of the sample:

- train set: 23,974 samples from 2008-2018 (B:11,987 M:11,987)
- test set 1: 1,622 samples from 2019 (B:1,453 M:169)
- test set 2: 1,291 samples from 2020 (B:1,041 M:250)

The category distribution in these sets is shown in Fig. 5. The ratio of malicious samples to benign samples in test set 1 and test set 2 is 1:8.62 and 1:4.16 respectively. This imbalance could cause a model to over-fit and affect the results. In future work, this imbalance should be addressed.

The four models are implemented (without feature selection) on a CPU-based setup. The libraries, modules, and

TABLE 6. Implementation details.

Model	Setup - Feature Selection	Module & Function	Details	Features
LR	1. CPU - No Feature Selection	sklearn.linear_model LogisticRegression()	solver = 'lbfgs' max_iter = 500	488 (S:322, D:166)
	1. CPU - Variance Threshold	sklearn.feature_selection	threshold=0.0117 VarianceThreshold()	88 (S: 23, D: 65)
	1. CPU - SelectFromModel	sklearn.feature_selection SelectFromModel	penalty='l1' solver='liblinear'	88 (S: 25, D: 63)
	2. GPU - No Feature Selection	cuml.linear_model	solver='qn' max_iter = 1000	488 (S:322, D:166)
SGD	1. CPU - No Feature Selection	sklearn.ensemble SGDClassifier()	random_state=42	488 (S:322, D:166)
	1. CPU - Variance Threshold	sklearn.feature_selection VarianceThreshold()	threshold=0.0117	88 (S:23, D:65)
	1. CPU - SelectFromModel	sklearn.feature_selection SelectFromModel()	-	88 (S:27 , D:61)
	2. GPU - No Feature Selection	cuml.linear_model MBSGDClassifier()	Epochs(100,150,200,1000) Batch_size (32,16,18,12) Eta0=(0.001, 0.003,0.004)	488 (S:322, D:166)
RF	1. CPU - No Feature Selection	sklearn.ensemble RandomForestClassifier()	n_estimators=55 random_state=42	488 (S:322, D:166)
	1. CPU - SelectFromModel	sklearn.feature_selection SelectFromModel()	-	88 (S:68 , D:20)
	1. CPU - Variance Threshold	sklearn.feature_selection VarianceThreshold()	threshold=0.01195	88 (S:24, D:64)
	2. GPU - No Feature Selection	cuml.ensemble RandomForestClassifier()	n_estimators=55	488 (S:322, D:166)
NN	1. CPU - No Feature Selection	TensorFlow CPU (tensorflow.keras.models) Sequential()	optimizer=adam loss=binary_crossentropy epochs=10 batch_size=32	488 (S:322, D:166)
	1. CPU - SelectKBest	sklearn.feature_selection SelectKBest()	score_function=chi ²	88 (S:32, D:56)
	1. CPU - Principal Component Analysis	sklearn.decomposition PrincipalComponentAnalysis()	-	81 Components
	2. GPU - No Feature Selection	TensorFlow GPU (tensorflow.keras.models) Sequential()	optimizer=adam loss=binary_crossentropy epochs=10 batch_size=32	488 (S: 322, D: 166)

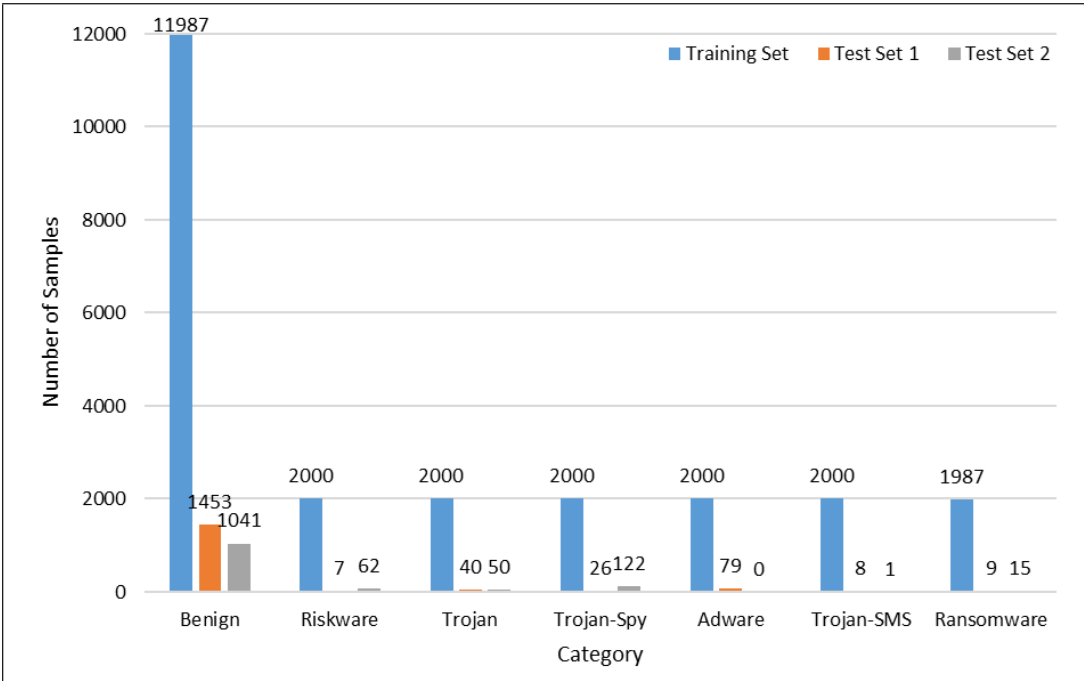


FIGURE 5. RO-2: Distribution of categories in dataset.

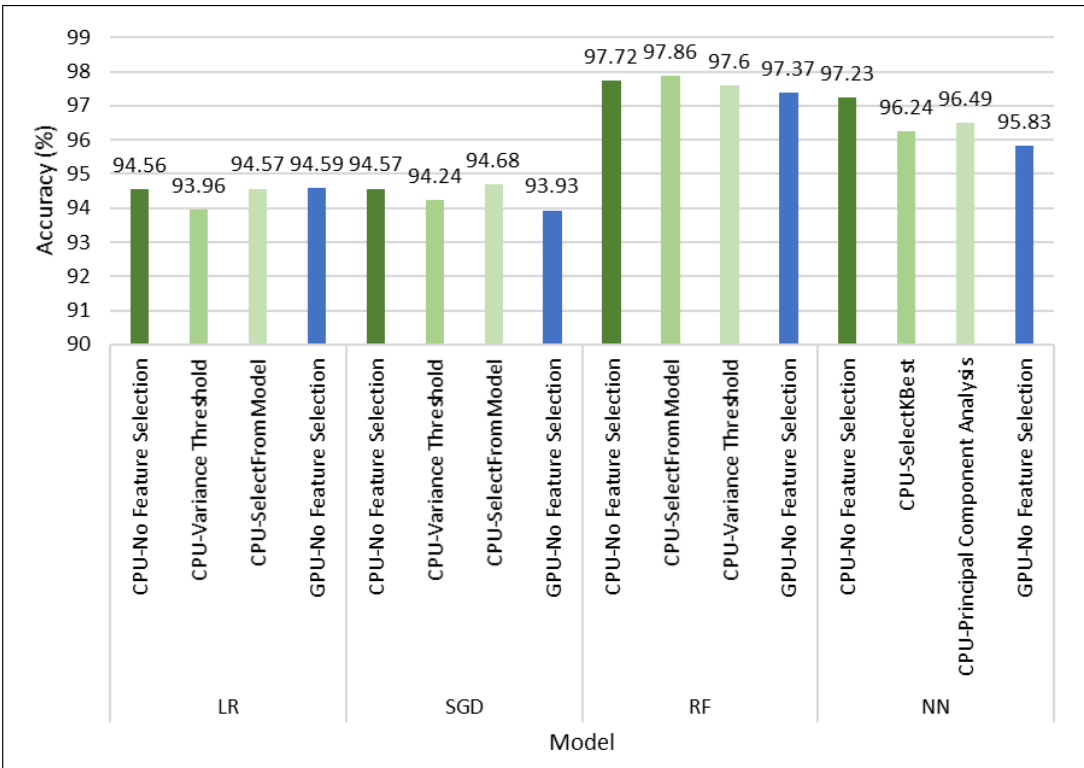


FIGURE 6. RO-1: Accuracy (%).

hyperparameters used are shown in Table 6 and the code is available at [67].

For model evaluation, the following four metrics are recorded:

- Accuracy (ACC)
- F1 Score

- False Positive Rate (FPR)
- Training Time (TT) in seconds - the time taken to train the model using the training set

The difference in ACC, FPR, and F1 score between test set 1 to test set 2, is used to evaluate the sustainability of a model.

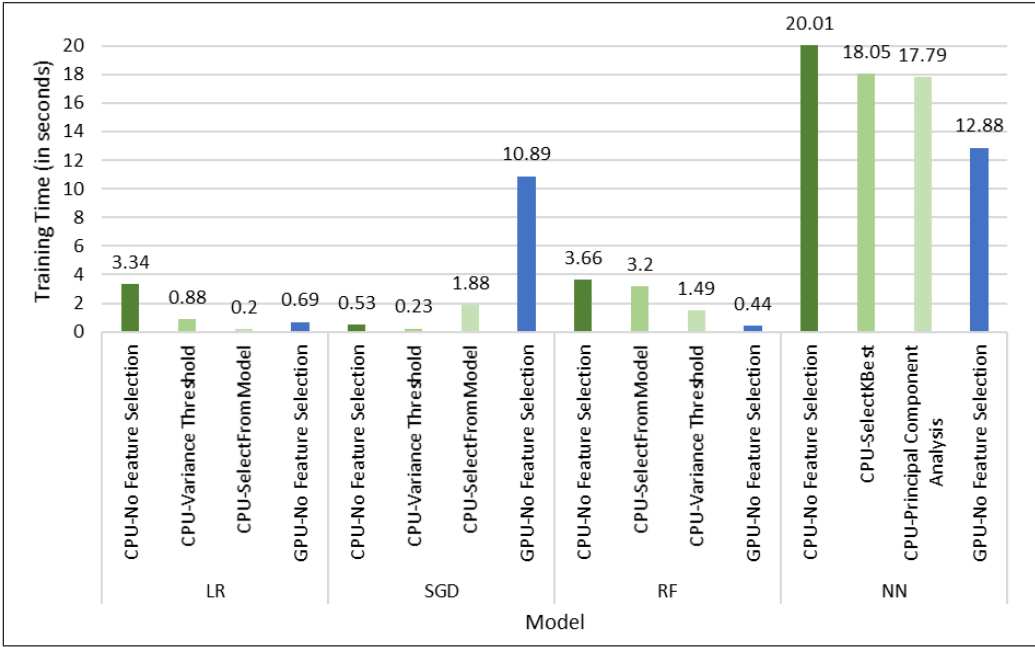


FIGURE 7. RO-1: Training-Time (s).

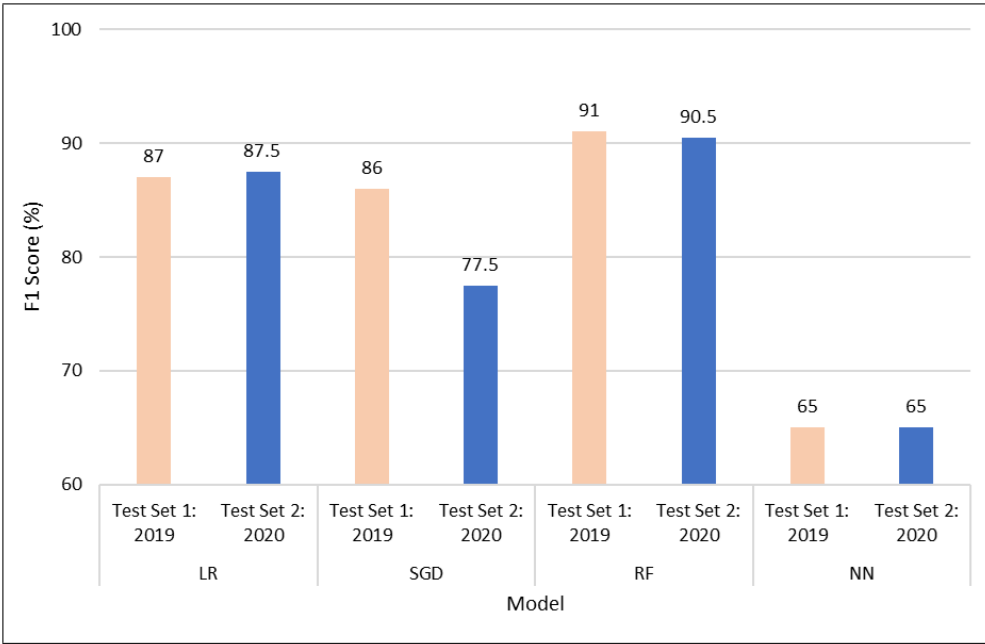


FIGURE 8. RO-2: F1-Score (%).

With regard to performance, the Training Time (TT) is used to quantify how long it takes to train an outdated model.

IV. RESULTS

A. RESULTS FOR RO-1

Table 7 presents the results of LR, RF, SGD, and NN models. The results for the most important metrics, (i.e. accuracy and training time) are shown in Fig. 6 and Fig. 7.

1) RESULTS FOR CPU-BASED VS. GPU-BASED SETUPS

It can be easily seen that the training time for LR, RF, and NN models improves significantly when a GPU-based setup is used instead of a CPU-based setup. The only exception is the SGD model. The use of MBSGDClassifier() function for the GPU setup, instead of the CPU-based SGDClassifier() function, leads to an increase of around 10 s. This highlights the need for improving ML libraries to take advantage of the computational resources provided by GPU-based setups.

TABLE 7. RO-1: Performance of models on CPU-based setup (No feature selection vs feature selection) and GPU-based setup.

Model	Setup - Feature Selection	MS (in MB)	TT (in s)	PT (in s)	ACC (%)	F1 Score (%)	FPR (%)
LR	1. CPU - No Feature Selection	0.004	3.34	0.020	94.56	95.00	3.8
	1. CPU - Variance Threshold	0.001*	0.88	0.005	93.96	93.50	4.0
	1. CPU - Regularization	0.001*	0.20*	0.004	94.57	94.50	3.8
	2. GPU - No Feature Selection	0.005	0.69	0.040	94.59	95.00	3.7
SGD	1. CPU - No Feature Selection	0.004	0.53	0.009	94.57	95.00	3.4
	1. CPU - Variance Threshold	0.001*	0.23	0.003*	94.24	94.50	4.6
	1. CPU - SelectFromModel	0.004	1.88	0.009	94.68	94.50	3.1
	2. GPU - No Feature Selection	0.005	10.89	0.014	93.93	94.00	4.9
RF	1. CPU - No Feature Selection	8.21	3.66	0.170	97.72	98.00*	1.5
	1. CPU - SelectFromModel	5.97	3.20	0.150	97.86*	98.00*	1.4*
	1. CPU - Variance Threshold	9.62	1.49	0.120	97.60	97.75	2.3
	2. GPU - No Feature Selection	6.54	0.44	0.570	97.37	98.00*	1.7
NN	1. CPU - No Feature Selection	0.09	20.01	1.100	97.23	97.00	2.2
	1. CPU - SelectKBest	0.05	18.05	1.050	96.24	96.00	3.8
	1. CPU - Principal Component Analysis	0.05	17.79	1.920	96.49	96.50	2.9
	2. GPU - No Feature Selection	0.08	12.88	0.760	95.83	96.00	2.6

2) RESULTS FOR CPU-BASED SETUP WITH VS. WITHOUT FEATURE SELECTION

For majority of the models, the use of feature selection results in shorter training time. The only exception is when the SGD model is used with SelectFromModel feature selection technique.

3) RESULTS FOR TRADE-OFF BETWEEN EFFECTIVENESS AND EFFICIENCY

The RF model, on a CPU-based setup (with variance threshold) has the highest accuracy of 97.86%, highest F1 score of 98%, and lowest FPR of 1.4%. The second-best

model, in terms of effectiveness, is also RF, but without any feature selection technique.

Meanwhile, the LR model (with regularization) has the smallest size (0.001 MB) and the lowest training time (0.20 s) compared to all the other models. Its prediction time is only 0.001 s slower than second best model i.e. SGD (with variance threshold).

The slowest training time of 20.01 s was recorded by the NN model (without feature selection) on a CPU-based setup. Although using a GPU-based setup helped to reduce training to 12.88 s, it was still substantially higher than the models.

TABLE 8. RO-2: Sustainability of LR, SGD, RF, and NN models.

Model	Test Set	ACC (%)	Change in ACC	FPR (%)	Change in FPR	F1 Score	Change in F1 Score
LR	2019	95.38	2.74	2.0	-1.1*	87.0	-0.5*
	2020	92.64		3.1		87.5	
SGD	2019	95.25	7.33	1.4*	-1.5	86.0	8.5
	2020	87.92		2.9		77.5	
RF	2019	96.61*	2.34*	2.0	-1.9	91.0*	0.5
	2020	94.27		3.9		90.5	
NN	2019	86.62	7.53	7.6	-3.4	65.0	0
	2020	79.09		11.0		65.0	

TABLE 9. RO-1: Comparison of performance (ACC) with existing studies with android datasets.

Source	Year	Dataset	Approach	LR		SGD		RF		NN	
				CPU	GPU	CPU	GPU	CPU	GPU	CPU	GPU
[23]	2022	Own	Static	-	-	-	-	89.96	-	-	-
[24]	2022	Drebin-215	Static	-	97.90	-	-	-	98.65	-	-
[28]	2023	Drebin	Static	-	-	98.63	-	99.23 ¹	-	-	-
		AndroZoo	Static	-	-	88.74	-	91.54 ¹	-	-	-
[22]	2023	KronoDroid	Hybrid (Emulator)	-	-	-	-	93	-	-	-
			Hybrid (Real)	-	-	-	-	95	-	-	-
Ours	2024	Enhanced Subset of KronoDroid	Hybrid (Real)	94.56	94.59	94.57	93.93	97.72*	97.37	97.23	95.83

¹ Adaptive RF

Overall, it can be noted that the most effective model is RF while the most efficient performance is obtained using simpler models like LR and SGD.

B. RESULTS FOR RO-2

Table 8 compares the sustainability of LR, SGD, RF, and NN models. Based on F1 score, the RF model is most sustainable with the highest value of 91% and a drop of only 0.5 % from 2019 to 2020 (see Fig. 8). The next most sustainable model is LR with a nominal increase in F1 score from

87% in 2019 to 87.5% in 2020. NN is the least sustainable model and this could partly be explained by NN being most affected by the imbalance in the number of malicious and benign samples in the two test sets as mentioned in section III-C.

V. DISCUSSION

A. DISCUSSION OF RO-1

Table 9 compares the findings with the accuracy reported in recent studies. Although the accuracy reported in [23],

TABLE 10. RO-2: Comparison of sustainability (Average F1 Score) with existing studies with android datasets.

Source	Year	Dataset	Approach	ML/DL Algorithm	Training Set Year(s)	Test Set Year(s)	Average F1 Score
[26]	2019	Own	Static	Online Learning	2011	2012-2016	89.97
					2011-2012	2013-2016	87.46
					2011-2013	2014-2016	87.17
					2011-2014	2015-2016	89.30
					2011-2015	2016	95.32
[27]	2020	Own	Dynamic (Emulator)	RF	2010	2011-2017	80-90
					2011	2012-2017	70-80
					2012	2013-2017	60-70
					2013	2014-2017	70-80
					2014	2015-2017	60-70
					2015	2016-2017	60-70
					2016	2017	70-80
[28]	2023	DREBIN	Static	A-RF ¹	2009-2012 ²	2009-2012 ³	73.26
		AndroZoo		SGD	2009-2012 ²	2009-2012 ³	75.18
				A-RF ¹	2016-2018 ²	2016-2018 ³	57.13
				SGD	2016-2018 ²	2016-2018 ³	52.79
Ours	2024	Enhanced	Hybrid	LR	2008-2018	2019-2020	87.25
		Subset of		SGD	2008-2018	2019-2020	81.75
		KronoDroid		RF	2008-2018	2019-2020	90.75*
				NN	2008-2018	2019-2020	65.00

¹ Adaptive RF² First Half of³ Second Half of

[24], and [28] is slightly higher than ours, it is vital to note these models utilise only static analysis. This approach is not effective for detecting sophisticated malware (especially obfuscated and polymorphic malware). Such malware should be analysed using a hybrid approach. The only study that uses this approach is [22]. Our highest accuracy of 97.78% is 2.78% higher than their reported accuracy of 95%.

A comparison with the studies listed in Table 1 is not possible because most of them do not utilise an Android dataset or they employ DL models that were not included in this study because of their significant computational requirements.

B. DISCUSSION OF RO-2

Table 10 compares the findings with the average F1 score reported in recent literature. The adoption of static approach by [26] and [28] means that these detectors will be less effective in detecting sophisticated malware. Similarly, the use of an emulator, instead of a real device, in [27] is a significant limitation. An emulator would fail to adequately examine malware that can change its behaviour when run in a virtual environment.

Our study overcomes these limitations by employing a hybrid analysis approach (in which dynamic analysis is performed on a real device) to obtain an average F1 score of 90.75% using RF.

VI. CONCLUSION

We propose the design of an effective, efficient, and sustainable malware detector based on empirical evidence. To ensure the detector is effective against sophisticated malware, we first create an enhanced subset of the publicly-available KronoDroid dataset (compiled via hybrid analysis of apps on a real device). Then we train and test LR, SGD, RF, and NN models on a CPU-based setup and a GPU-based setup. We also apply two different feature selection techniques to determine the effect on performance. Next, to assess sustainability, we split the dataset based on the year of the sample app. The same four models are trained using samples from 2008-2018 and tested using samples from 2019 and 2020.

Our findings show that the training time of RF and NN models are significantly reduced when a GPU-based setup is used instead of a CPU-based setup. Likewise, we find that the use of feature selection also helps reduce training time of all models except SGD. In both cases, the exceptions could be explained by the optimisation of libraries (for GPU-based setup) for processing huge feature sets and deep learning algorithms.

Overall, the most efficient model, in terms of resource-usage, is LR (using regularisation) followed closely by SGD (using variance threshold). On the other hand, the most effective model is RF (using SelectFromModel), with an accuracy of 97.86% and a FPR of 1.4%. This accuracy is 2.78% higher than the highest accuracy reported in existing literature. Also, the RF model proves to be the most sustainable with minimal change in F1 score between test set 1 (2019) and test set 1 (2020).

The key takeaway is that empirical evidence shows that simple classifiers, like LR and RF, are most suitable for development of real-time, effective, and sustainable malware detectors. Additionally, the potential for sophisticated features selection techniques and improvements in libraries for GPU-based setups should be explored as promising avenues for more efficient performance.

As future work, some of the limitation of this paper could be overcome. One important example is the imbalance in samples belonging to each year. This could best be overcome by integrating malware analysis and detection processes into SIEM (Security Information and Event Management) systems and XDR (Extended Detection and Response) platforms. Such automation would speed up sample collection and analysis. Another useful improvement for incident response and security and malware analyst could be the inclusion of eXplainable AI (XAI) to provide an interpretation of prediction thereby enhancing trust and improving model transparency.

REFERENCES

- [1] F. A. Narudin, A. Feizollah, N. B. Anuar, and A. Gani, "Evaluation of machine learning classifiers for mobile malware detection," *Soft Comput.*, vol. 20, no. 1, pp. 343–357, Jan. 2016, doi: [10.1007/s00500-014-1511-6](https://doi.org/10.1007/s00500-014-1511-6).
- [2] B. Cozens. (Mar. 1, 2023). *Crushing the Two Biggest Threats to Mobile Endpoint Security in 2023*. [Online]. Available: <https://www.malwarebytes.com/blog/business/2023/03/crushing-the-two-biggest-threats-to-mobile-endpoint-security-in-2023>
- [3] Zimperium. (2023). *2023 Global Mobile Threat Report*. [Online]. Available: <https://get.zimperium.com/2023-global-mobile-threat-report>
- [4] M. Motylinski, Á. MacDermott, F. Iqbal, and B. Shah, "A GPU-based machine learning approach for detection of botnet attacks," *Comput. Secur.*, vol. 123, Dec. 2022, Art. no. 102918, doi: [10.1016/j.cose.2022.102918](https://doi.org/10.1016/j.cose.2022.102918).
- [5] Ö. A. Aslan and R. Samet, "A comprehensive review on malware detection approaches," *IEEE Access*, vol. 8, pp. 6249–6271, 2020, doi: [10.1109/ACCESS.2019.2963724](https://doi.org/10.1109/ACCESS.2019.2963724).
- [6] C. Willems, T. Holz, and F. Freiling, "Toward automated dynamic malware analysis using CWSandbox," *IEEE Secur. Privacy*, vol. 5, no. 2, pp. 32–39, Mar. 2007, doi: [10.1109/MSP.2007.45](https://doi.org/10.1109/MSP.2007.45).
- [7] J. Jeon, J. H. Park, and Y.-S. Jeong, "Dynamic analysis for IoT malware detection with convolution neural network model," *IEEE Access*, vol. 8, pp. 96899–96911, 2020, doi: [10.1109/ACCESS.2020.2995887](https://doi.org/10.1109/ACCESS.2020.2995887).
- [8] K. Sethi, R. Kumar, L. Sethi, P. Bera, and P. K. Patra, "A novel machine learning based malware detection and classification framework," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Services (Cyber Security)*, Oxford, U.K., Jun. 2019, pp. 1–4, doi: [10.1109/CyberSecPODS.2019.8885196](https://doi.org/10.1109/CyberSecPODS.2019.8885196).
- [9] L. Liu, B.-S. Wang, B. Yu, and Q.-X. Zhong, "Automatic malware classification and new malware detection using machine learning," *Frontiers Inf. Technol. Electron. Eng.*, vol. 18, no. 9, pp. 1336–1347, Sep. 2017, doi: [10.1631/jitee.1601325](https://doi.org/10.1631/jitee.1601325).
- [10] D. Uppal, R. Sinha, V. Mehra, and V. Jain, "Malware detection and classification based on extraction of API sequences," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Delhi, India, Sep. 2014, pp. 2337–2342, doi: [10.1109/ICACCI.2014.6968547](https://doi.org/10.1109/ICACCI.2014.6968547).
- [11] D. Bekerman, B. Shapira, L. Rokach, and A. Bar, "Unknown malware detection using network traffic classification," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Florence, Italy, Sep. 2015, pp. 134–142, doi: [10.1109/CNS.2015.7346821](https://doi.org/10.1109/CNS.2015.7346821).
- [12] M. Norouzi, A. Sour, and M. S. Zamini, "A data mining classification approach for behavioral malware detection," *J. Comput. Netw. Commun.*, vol. 2, pp. 1–9, Jan. 2016, doi: [10.1155/2016/8069672](https://doi.org/10.1155/2016/8069672).
- [13] R. Sihwail, K. Omar, and K. A. Z. Ariffin, "An effective memory analysis for malware detection and classification," *Comput., Mater. Continua*, vol. 67, no. 2, pp. 2301–2320, 2021, doi: [10.32604/cmc.2021.014510](https://doi.org/10.32604/cmc.2021.014510).
- [14] K. Sethi, S. K. Chaudhary, B. K. Tripathy, and P. Bera, "A novel malware analysis for malware detection and classification using machine learning algorithms," in *Proc. 10th Int. Conf. Secur. Inf. Netw.*, Jaipur, India, Oct. 2017, pp. 107–113, doi: [10.1145/3136825.3136883](https://doi.org/10.1145/3136825.3136883).
- [15] H. S. Anderson, A. Kharkar, B. Filar, and P. Roth. (2017). *Evading Machine Learning Malware Detection*. BlackHat, Las Vegas, NV, USA. [Online]. Available: <https://www.blackhat.com/docs/us-17/thursday/us-17-Anders-on-Bot-Vs-Bot-Evading-Machine-Learning-Malware-Detection-wp.pdf>
- [16] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "Machine learning and deep learning approaches for cybersecurity: A review," *IEEE Access*, vol. 10, pp. 19572–19585, 2022, doi: [10.1109/ACCESS.2022.3151248](https://doi.org/10.1109/ACCESS.2022.3151248).
- [17] A. Abusnaina, M. Abuhamad, H. Alasmay, A. Anwar, R. Jang, S. Salem, D. Nyang, and D. Mohaisen, "DL-FHMC: Deep learning-based fine-grained hierarchical learning approach for robust malware classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3432–3447, Sep. 2022, doi: [10.1109/TDSC.2021.3097296](https://doi.org/10.1109/TDSC.2021.3097296).
- [18] X. Chen, Z. Hao, L. Li, L. Cui, Y. Zhu, Z. Ding, and Y. Liu, "CruParamer: Learning on parameter-augmented API sequences for malware detection," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 788–803, 2022, doi: [10.1109/TIFS.2022.3152360](https://doi.org/10.1109/TIFS.2022.3152360).
- [19] A. A. Al-Hashmi, F. A. Ghaleb, A. Al-Marghilani, A. E. Yahya, S. A. Ebad, M. S. M. Saqib, and A. A. Darem, "Deep-ensemble and multifaceted behavioral malware variant detection model," *IEEE Access*, vol. 10, pp. 42762–42777, 2022, doi: [10.1109/ACCESS.2022.3168794](https://doi.org/10.1109/ACCESS.2022.3168794).
- [20] Y.-S. Jeong, S.-M. Lee, J.-H. Kim, J. Woo, and A. R. Kang, "Malware detection using byte streams of different file formats," *IEEE Access*, vol. 10, pp. 51041–51047, 2022, doi: [10.1109/ACCESS.2022.3171775](https://doi.org/10.1109/ACCESS.2022.3171775).
- [21] Y. Xin, L. Kong, Z. Liu, Y. Chen, Y. Li, H. Zhu, M. Gao, H. Hou, and C. Wang, "Machine learning and deep learning methods for cybersecurity," *IEEE Access*, vol. 6, pp. 35365–35381, 2018, doi: [10.1109/ACCESS.2018.2836950](https://doi.org/10.1109/ACCESS.2018.2836950).

- [22] S. Aurangzeb and M. Aleem, "Evaluation and classification of obfuscated Android malware through deep learning using ensemble voting mechanism," *Sci. Rep.*, vol. 13, no. 1, p. 3093, Feb. 2023, doi: [10.1038/s41598-023-30028-w](https://doi.org/10.1038/s41598-023-30028-w).
- [23] F. Akbar, M. Hussain, R. Mumtaz, Q. Riaz, A. W. A. Wahab, and K.-H. Jung, "Permissions-based detection of Android malware using machine learning," *Symmetry*, vol. 14, no. 4, p. 718, Apr. 2022, doi: [10.3390/sym14040718](https://doi.org/10.3390/sym14040718).
- [24] M. M. Alani and A. I. Awad, "PAIRED: An explainable lightweight Android malware detection system," *IEEE Access*, vol. 10, pp. 73214–73228, 2022, doi: [10.1109/ACCESS.2022.3189645](https://doi.org/10.1109/ACCESS.2022.3189645).
- [25] E. C. Bayazit, O. K. Sahingoz, and B. Dogan, "Neural network based Android malware detection with different IP coding methods," in *Proc. 3rd Int. Congr. Hum.-Comput. Interact., Optim. Robotic Appl. (HORA)*, Ankara, Turkey, Jun. 2021, pp. 1–6, doi: [10.1109/HORA52670.2021.9461302](https://doi.org/10.1109/HORA52670.2021.9461302).
- [26] K. Xu, Y. Li, R. Deng, K. Chen, and J. Xu, "DroidEvolver: Self-evolving Android malware detection system," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Stockholm, Sweden, Jun. 2019, pp. 47–62, doi: [10.1109/EUROSP.2019.00014](https://doi.org/10.1109/EUROSP.2019.00014).
- [27] H. Cai, "Assessing and improving malware detection sustainability through app evolution studies," *ACM Trans. Softw. Eng. Methodol.*, vol. 29, no. 2, pp. 1–28, Apr. 2020, doi: [10.1145/3371924](https://doi.org/10.1145/3371924).
- [28] F. Ceschin, M. Botacin, H. M. Gomes, F. Pinagé, L. S. Oliveira, and A. Grégio, "Fast & furious: On the modelling of malware detection as an evolving data stream," *Expert Syst. Appl.*, vol. 212, Feb. 2023, Art. no. 118590, doi: [10.1016/j.eswa.2022.118590](https://doi.org/10.1016/j.eswa.2022.118590).
- [29] S. Rastegari, "Intelligent network intrusion detection using an evolutionary computation approach," Ph.D. thesis, Dept. Comput. Secur. Sci., Edith Cowan Univ., Joondalup, WA, Australia, 2015. [Online]. Available: <https://ro.ecu.edu.au/cgi/viewcontent.cgi?article=2761&context=theses>
- [30] Kaspersky. (2021). *Machine Learning for Malware Detection*. [Online]. Available: <https://media.kaspersky.com/en/enterprise-security/Kaspersky-Lab-Whitepaper-Machine-Learning.pdf>
- [31] M. Shafiq, Z. Tian, Y. Sun, X. Du, and M. Guizani, "Selection of effective machine learning algorithm and bot-IoT attacks traffic identification for Internet of Things in smart city," *Future Generat. Comput. Syst.*, vol. 107, pp. 433–442, Jun. 2020, doi: [10.1016/j.future.2020.02.017](https://doi.org/10.1016/j.future.2020.02.017).
- [32] M. M. Koushki, I. Abualhaol, A. D. Raju, Y. Zhou, R. S. Giagone, and H. Shengqiang, "On building machine learning pipelines for Android malware detection: A procedural survey of practices, challenges and opportunities," *Cybersecurity*, vol. 5, no. 1, p. 16, Dec. 2022, doi: [10.1186/s42400-022-00119-8](https://doi.org/10.1186/s42400-022-00119-8).
- [33] B. Yuan, J. Wang, P. Wu, and X. Qing, "IoT malware classification based on lightweight convolutional neural networks," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3770–3783, Mar. 2022, doi: [10.1109/JIOT.2021.3100063](https://doi.org/10.1109/JIOT.2021.3100063).
- [34] X. Xing, X. Jin, H. Elahi, H. Jiang, and G. Wang, "A malware detection approach using autoencoder in deep learning," *IEEE Access*, vol. 10, pp. 25696–25706, 2022, doi: [10.1109/ACCESS.2022.3155695](https://doi.org/10.1109/ACCESS.2022.3155695).
- [35] P. Manirihio, A. N. Mahmood, and M. J. M. Chowdhury, "Deep learning models for detecting malware attacks," 2022, *arXiv:2209.03622*.
- [36] O. O. Petrinir, F. Saeed, X. Li, F. Ghabban, and K.-C. Wong, "Malicious traffic detection in IoT and local networks using stacked ensemble classifier," *Comput., Mater. Continua*, vol. 71, no. 1, pp. 489–515, 2022, doi: [10.32604/cmc.2022.019636](https://doi.org/10.32604/cmc.2022.019636).
- [37] A. Djenna, A. Bouridane, S. Rubab, and I. M. Marou, "Artificial intelligence-based malware detection, analysis, and mitigation," *Symmetry*, vol. 15, no. 3, p. 677, Mar. 2023, doi: [10.3390/sym15030677](https://doi.org/10.3390/sym15030677).
- [38] Z. Ma, H. Ge, Y. Liu, M. Zhao, and J. Ma, "A combination method for Android malware detection based on control flow graphs and machine learning algorithms," *IEEE Access*, vol. 7, pp. 21235–21245, 2019, doi: [10.1109/ACCESS.2019.2896003](https://doi.org/10.1109/ACCESS.2019.2896003).
- [39] D. Ö. Sahin, S. Akleyek, and E. Kiliç, "LinRegDroid: Detection of Android malware using multiple linear regression models-based classifiers," *IEEE Access*, vol. 10, pp. 14246–14259, 2022, doi: [10.1109/ACCESS.2022.3146363](https://doi.org/10.1109/ACCESS.2022.3146363).
- [40] R. S. Pircoveanu, S. S. Hansen, T. M. T. Larsen, M. Stevanovic, J. M. Pedersen, and A. Czech, "Analysis of malware behavior: Type classification using machine learning," in *Proc. Int. Conf. Cyber Situational Awareness, Data Anal. Assessment (CyberSA)*, London, U.K., Jun. 2015, pp. 1–7, doi: [10.1109/CyberSA.2015.7166115](https://doi.org/10.1109/CyberSA.2015.7166115).
- [41] N. Milosevic, A. Dehghantanha, and K.-K.-R. Choo, "Machine learning aided Android malware classification," *Comput. Electr. Eng.*, vol. 61, pp. 266–274, Jul. 2017, doi: [10.1016/j.compeleceng.2017.02.013](https://doi.org/10.1016/j.compeleceng.2017.02.013).
- [42] G. Suarez-Tangil, S. K. Dash, M. Ahmadi, J. Kinder, G. Giacinto, and L. Cavallaro, "DroidSieve: Fast and accurate classification of obfuscated Android malware," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, Mar. 2017, pp. 309–320, doi: [10.1145/3029806.3029825](https://doi.org/10.1145/3029806.3029825).
- [43] K. Sethi, S. K. Chaudhary, B. K. Tripathy, and P. Bera, "A novel malware analysis framework for malware detection and classification using machine learning approach," in *Proc. 19th Int. Conf. Distrib. Comput. Netw.*, Varanasi, India, Jan. 2018, pp. 1–4, doi: [10.1145/3154273.3154326](https://doi.org/10.1145/3154273.3154326).
- [44] M. Waheed and S. Qadir, "Effective and efficient Android malware detection and category classification using the enhanced KronoDroid dataset," *Secur. Commun. Netw.*, vol. 1, Apr. 2024, Art. no. 7382302, doi: [10.1155/2024/7382302](https://doi.org/10.1155/2024/7382302).
- [45] R. Casolare, C. D. Dominici, G. Iadarola, F. Martinelli, F. Mercaldo, and A. Santone, "Dynamic mobile malware detection through system call-based image representation," *J. Wireless Mobile Netw., Ubiquitous Comput., Dependable Appl. (JoWUA)*, vol. 12, no. 1, pp. 44–63, 2021, doi: [10.22667/JOWUA.2021.03.31.044](https://doi.org/10.22667/JOWUA.2021.03.31.044).
- [46] K. Bakour and H. M. Ünver, "VisDroid: Android malware classification based on local and global image features, bag of visual words and machine learning techniques," *Neural Comput. Appl.*, vol. 33, no. 8, pp. 3133–3153, Apr. 2021, doi: [10.1007/s00521-020-05195-w](https://doi.org/10.1007/s00521-020-05195-w).
- [47] Q.-D. Ngo, H.-T. Nguyen, V.-H. Le, and D.-H. Nguyen, "A survey of IoT malware and detection methods based on static features," *ICT Exp.*, vol. 6, no. 4, pp. 280–286, Dec. 2020, doi: [10.1016/j.ict.2020.04.005](https://doi.org/10.1016/j.ict.2020.04.005).
- [48] S. K. Sahay and M. Chaudhari, "An efficient detection of malware by naive Bayes classifier using GPGPU," 2019, *arXiv:1905.13746*.
- [49] S. Sharma, C. R. Krishna, and R. Kumar, "Android ransomware detection using machine learning techniques: A comparative analysis on GPU and CPU," in *Proc. 21st Int. Arab Conf. Inf. Technol. (ACIT)*, Giza, Egypt, Nov. 2020, pp. 1–6, doi: [10.1109/ACIT50332.2020.9300108](https://doi.org/10.1109/ACIT50332.2020.9300108).
- [50] N. Bakshshinejad and A. Hamzeh, "Parallel-CNN network for malware detection," *IET Inf. Secur.*, vol. 14, no. 2, pp. 210–219, Mar. 2020, doi: [10.1049/iet-ifs.2019.0159](https://doi.org/10.1049/iet-ifs.2019.0159).
- [51] J. Jeon, J. Kim, S. Jeon, S. Lee, and Y.-S. Jeong, "Static analysis for malware detection with tensorflow and GPU," *Adv. Comput. Sci. Ubiquitous Comput.*, pp. 537–546, Jan. 2021, doi: [10.1007/978-981-15-9343-7](https://doi.org/10.1007/978-981-15-9343-7).
- [52] V. Hnamte and J. Hussain, "Dependable intrusion detection system using deep convolutional neural network: A novel framework and performance evaluation approach," *Telematics Informat. Rep.*, vol. 11, Sep. 2023, Art. no. 100077, doi: [10.1016/j.teler.2023.100077](https://doi.org/10.1016/j.teler.2023.100077).
- [53] M. Waqar, S. Fareed, A. Kim, S. Ur Rehman Malik, M. Imran, and M. U. Yaseen, "Malware detection in Android IoT systems using deep learning," *Comput., Mater. Continua*, vol. 74, no. 2, pp. 4399–4415, 2023, doi: [10.32604/cmc.2023.032984](https://doi.org/10.32604/cmc.2023.032984).
- [54] H. Cai and B. Ryder, "A longitudinal study of application structure and behaviors in Android," *IEEE Trans. Softw. Eng.*, vol. 47, no. 12, pp. 2934–2955, Dec. 2021, doi: [10.1109/TSE.2020.2975176](https://doi.org/10.1109/TSE.2020.2975176).
- [55] H. Cai, X. Fu, and A. Hamou-Lhadj, "A study of run-time behavioral evolution of benign versus malicious apps in Android," *Inf. Softw. Technol.*, vol. 122, Jun. 2020, Art. no. 106291, doi: [10.1016/j.infsof.2020.106291](https://doi.org/10.1016/j.infsof.2020.106291).
- [56] G. Suarez-Tangil and G. Stringhini, "Eight years of rider measurement in the Android malware ecosystem," *IEEE Trans. Dependable Secure Comput.*, vol. 1, no. 1, pp. 107–118, Jan./Feb. 2022, doi: [10.1109/TDSC.2022.3100063](https://doi.org/10.1109/TDSC.2022.3100063).
- [57] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Comput. Secur.*, vol. 81, pp. 123–147, Mar. 2019, doi: [10.1016/j.cose.2018.11.001](https://doi.org/10.1016/j.cose.2018.11.001).
- [58] A. Guerra-Manzanares, H. Bahsi, and S. Nömm, "KronoDroid: Time-based hybrid-featured dataset for effective Android malware detection and characterization," *Comput. Secur.*, vol. 110, Nov. 2021, Art. no. 102399, doi: [10.1016/j.cose.2021.102399](https://doi.org/10.1016/j.cose.2021.102399).
- [59] A. Guerra-Manzanares, M. Luckner, and H. Bahsi, "Android malware concept drift using system calls: Detection, characterization and challenges," *Expert Syst. Appl.*, vol. 206, Nov. 2022, Art. no. 117200, doi: [10.1016/j.eswa.2022.117200](https://doi.org/10.1016/j.eswa.2022.117200).
- [60] SEMW. (2022). *Semw/Kronodroid_Improved_Hybrid_Detection_V2*. [Online]. Available: https://github.com/semw/kronodroid_improved_hybrid_detection_v2

- [61] A. S. Shatnawi, A. Jaradat, T. B. Yaseen, E. Taqieddin, M. Al-Ayyoub, and D. Mustafa, "An Android malware detection leveraging machine learning," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, May 2022, Art. no. 1830201, doi: [10.1155/2022/1830201](https://doi.org/10.1155/2022/1830201).
- [62] S. Biswas, M. Bordoloi, and B. Purkayastha, "Review on feature selection and classification using neuro-fuzzy approaches," *Int. J. Appl. Evol. Comput.*, vol. 7, no. 4, pp. 28–44, Oct. 2016, doi: [10.4018/ijaec.2016100102](https://doi.org/10.4018/ijaec.2016100102).
- [63] Scikit-Learn. *Feature Selection-Scikit-Learn 1.5.1 Documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/feature_selection.html
- [64] A. Rahali, A. H. Lashkari, G. Kaur, L. Taheri, F. Gagnon, and F. Massicotte, "DIDroid: Android malware classification and characterization using deep image learning," in *Proc. 10th Int. Conf. Commun. Netw. Secur.*, Nov. 2020, pp. 70–82, doi: [10.1145/3442520.3442522](https://doi.org/10.1145/3442520.3442522).
- [65] A. R. Javed, Z. Jalil, S. A. Moqurrah, S. Abbas, and X. Liu, "Ensemble AdaBoost classifier for accurate and fast detection of botnet attacks in connected vehicles," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 10, p. e4088, Oct. 2022, doi: [10.1002/ett.4088](https://doi.org/10.1002/ett.4088).
- [66] M. K. Alzaylaee, S. Y. Yerima, and S. Sezer, "DL-Droid: Deep learning based Android malware detection using real devices," *Comput. Secur.*, vol. 89, Feb. 2020, Art. no. 101663, doi: [10.1016/j.cose.2019.101663](https://doi.org/10.1016/j.cose.2019.101663).
- [67] devNowRO. (2024). *DevNowRO/Enhanced-Subset-KronoDroid*. [Online]. Available: <https://github.com/devNowRO/Enhanced-Subset-KronoDroid>



development experience both in industry and in academia in the area of network security, malware detection, and applied cryptography.

SANA QADIR (Member, IEEE) received the bachelor's degree (Hons.) in information technology from the University of Southern Queensland, Australia, and the M.Sc. degree in computer and information engineering and the Ph.D. degree in engineering from International Islamic University Malaysia (IIUM), in 2010 and 2016, respectively. She is currently an Assistant Professor with the School of Electrical Engineering and Computer Science (SEECs), NUST. She has research and



AMNA NAEEM received the M.Sc. degree in computational science and engineering from the National University of Sciences and Technology (NUST), in 2023. She is currently a Research Associate with the School of Interdisciplinary Engineering and Sciences (SINES), NUST. Her primary research interests include machine learning, virtual and augmented reality, and artificial intelligence.



MEHDI HUSSAIN (Senior Member, IEEE) received the Ph.D. degree in information security from the University of Malaya, Malaysia, in 2017. Currently, he holds the position of an Associate Professor, the HoD of information security, and the Co-Director of the WALEE-NUST Laboratory, School of Electrical Engineering and Computer Science, National University of Sciences and Technology (NUST), Pakistan. With over 17 years of research and teaching experience, he has developed various real-time products and libraries. He is recognized as an approved HEC Ph.D. Supervisor. He has authored more than 35 publications in image steganography, network traffic analysis, and cybersecurity fields. His contributions to academia and industry can be explored further through his Google Scholar profile. His area of research interests include multimedia security, multimedia forensics, image steganography, image tamper detection, reversible data hiding, and video/image compression. He is a guest editor and a reviewer of prestigious journals.



HUMA GHAFOR received the B.Sc. degree in electrical engineering from the NFC Institute of Engineering and Technology, Multan, Pakistan, in 2009, the M.S. degree in communication systems engineering from the School of Electrical Engineering and Computer Science (SEECs), NUST, Islamabad, Pakistan, in 2012, and the Ph.D. degree in electrical engineering from the School of Electrical Engineering, University of Ulsan, South Korea, in 2018. She is currently an Assistant Professor with the School of Electrical Engineering and Computer Science, NUST. Until January 2019, she was a Postdoctoral Researcher with the Networked Systems Laboratory, Inha University, South Korea. Her research interests include vehicular ad hoc networks, maritime networks, underwater sensor networks, software-defined networking, and blockchain technology, with a focus on cognitive networking protocols.



AISHA HASSAN ABDALLA HASHIM (Senior Member, IEEE) received the B.Sc. degree in electronics engineering, in 1990, the M.Sc. degree in computer science, in 1996, and the Ph.D. degree in computer engineering, in 2007. She joined International Islamic University Malaysia (IIUM), in 1997, where she is currently a Professor with the Department of Electrical and Computer Engineering. She is actively involved in research and postgraduate programs. She has also secured research grants from IIUM; Ministry of Higher Education (MOHE); and the Ministry of Science, Technology and Innovation (MOSTI), Malaysia. She has published more than 200 journal/conference papers, and supervised/co-supervised more than 40 Ph.D./master's students. She has actively contributed as a reviewer/a technical committee member of many journals/conferences.

...